2ndProject

심규명

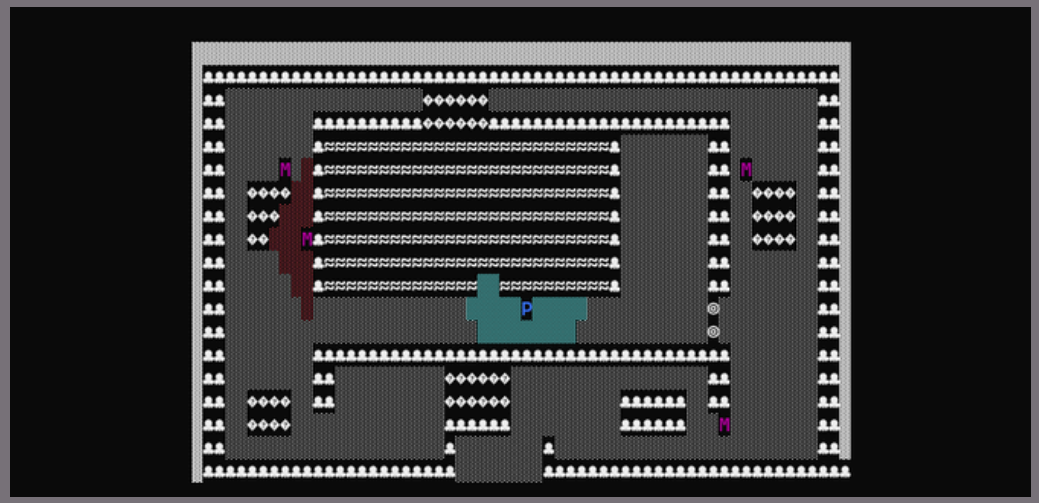
26.03.04 ~ 26.03.10

Contents

프로젝트 개요
개발 일정
구현, 트러블 슈팅
회고

프로젝트 개요

- 턴제 게임 → srpg 로 전환
- 타이틀-마을-필드-전투로 진행되는 사이클



개발 일정

3/4	**기획**	1.기존 완성 전투 로직 개선 2.필드 로직 + 전투 로직 => srpg 3.마을 필드 던전 순환 구조 설계
3/5 ~3/6	개발	기초 설계 : level 전환, json data, tilemap
3/7		BattleLevel 연결, 엔진 누수 방지
3/8		ANSI추가, Renderer 추가, 상태 이상 계산 추가
3/9		Actor 이동 시야 범위, 턴 기반 전투 상태 개선
3/10		Event 추가, 코드 리팩토링

기술 구현

길 찾기

```
struct Node
{
    Vector2 pos; //위치
    float g, h; //시작-내위치, 내 위치- 목적지
    Node* parent; //부모
    float f() const { return g + h; } //총 비용
};
```

0,0 → 4,4 목적지를 간다면

1,1에 도착한 상황

부모 노드: (0,0), g=0, h=8, f=8
- OpenList에 추가: {(0,0)}
- ClosedList: {}
- 목표지점 pos 비교 → 아직 목표 아님
- f값 최소 노드 선택 → 현재 노드 = (0,0)
- 4방향 탐색: (0,1), (1,0)
- 유효한 2개 노드 생성:
- (0,1), g=1, h=7, f=8, 부모=(0,0)
- (1,0), g=1, h=7, f=8, 부모=(0,0)
- OpenList 업데이트: {(0,1), (1,0)}

f값 최소 노드 선택 (0,1) (상하좌우 순서)
- 부모 노드: (0,1)
- ClosedList에 추가: {(0,0),(0,1)}
- 목표지점 pos 비교 → 목표 아님
- 4방향 탐색: (0,0), (0,2), (1,1)
- 유효 노드 생성:
- (0,2), g=2, h=6, f=8, 부모=(0,1)
- (1,1), g=2, h=6, f=8, 부모=(0,1)

f값 최소 노드 선택 → (1,0)
- 부모 노드: (1,0)
- ClosedList에 추가: {(0,0),(0,1),(1,0)}
- 목표지점 pos 비교 → 목표 아님
- 4방향 탐색: (0,0), (2,0), (1,1)
- 유효 노드 생성:
- (2,0), g=2, h=6, f=8, 부모=(1,0)
- (1,1) → 이미 OpenList에 존재
- OpenList 업데이트: {(0,2), (1,1), (2,0)}

f값 최소 노드 선택 → (0,2)
- 부모 노드: (0,2)
- ClosedList에 추가: {(0,0),(0,1),(1,0),(0,2)}
- 목표지점 pos 비교 → 목표 아님
- 4방향 탐색: (0,1), (0,3), (1,2)
- 유효 노드 생성:
- (0,3), g=3, h=5, f=8, 부모=(0,2)
- (1,2), g=3, h=5, f=8, 부모=(0,2)
- OpenList 업데이트: {(1,1), (2,0), (0,3), (1,2)}

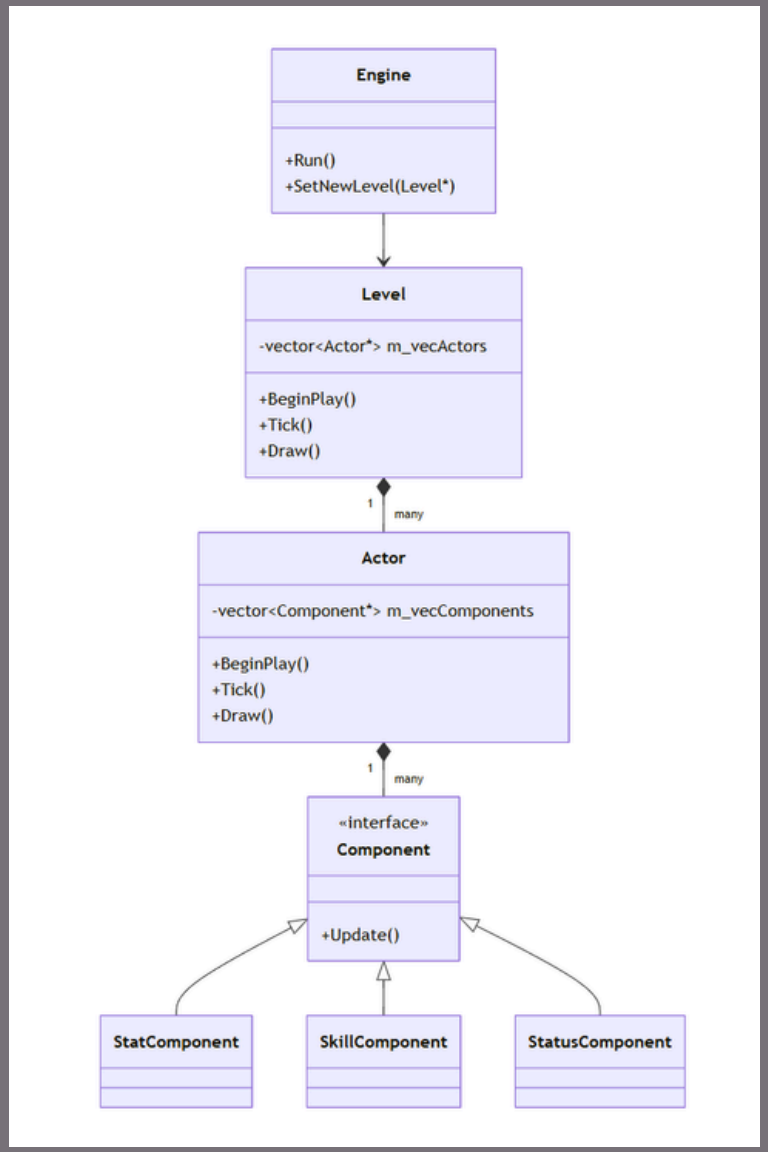
f값 최소 노드 선택 → (1,1)
- 부모 노드: (1,1)
- ClosedList에 추가: {(0,0),(0,1),(1,0),(0,2),(1,1)}
- 목표지점 pos 비교 → 목표 아님
- 4방향 탐색: (0,1), (1,0), (1,2), (2,1)
- 유효 노드 생성:
- (2,1), g=3, h=5, f=8, 부모=(1,1)
- (1,2) → 이미 OpenList에 존재, 기존 g=3 → 갱신 없음
- OpenList 업데이트: {(2,0), (0,3), (1,2), (2,1)}

기술 구현

```
std::vector<Node*> openList;
std::vector<Node*> closedList;

Vector2 startTemp;
Node* startNode = new Node{ start, 0, startTemp.Distance(v1: start, v2: end), nullptr }; //부모 노드
openList.emplace_back(& _Val: startNode);

while (!openList.empty())
{
    // F값이 가장 낮은 노드 선택
    auto std::vector<Node *>::iterator it = std::min_element(_First: openList.begin(), _Last: openList.end(),
        _Pred: [](Node* a, Node* b) { return a->f() < b->f(); });

    Node* current = *it;
    if (current->pos == end)
    {
        while (current)
        {
            path.emplace_back(& _Val: current->pos);
            current = current->parent;
        }
        std::reverse(_First: path.begin(), _Last: path.end());
        break;
    }
```

```
openList.erase(_Where: it);
closedList.emplace_back(& _Val: current);

Vector2 neighbors[] = { {0,-1}, {0,1}, {-1,0}, {1,0} }; //시계방향으로 체크
for (auto& Wannabe::Vector2 & dir : neighbors)
{
    Vector2 nextPos = current->pos + dir;
    if (!IsWalkable((int)nextPos.x, (int)nextPos.y)) continue;

    auto std::vector<Node *>::iterator isClosed = std::find_if(_First: closedList.begin(), _Last: closedList.end(),
        _Pred: [&](Node* n) { return n->pos == nextPos; });
    if (isClosed != closedList.end()) continue;

    float newG = current->g + 1;
    auto std::vector<Node *>::iterator isOpen = std::find_if(_First: openList.begin(), _Last: openList.end(),
        _Pred: [&](Node* n) { return n->pos == nextPos; });

    Vector2 temp;
    if (isOpen == openList.end())
    {
        openList.emplace_back(_Val: new Node{ nextPos, newG,  temp.Distance(v1: nextPos, v2: end), current });
    }
    else if (newG < (*isOpen)->g)
    {
        (*isOpen)->g = newG;
        (*isOpen)->parent = current;
    }
```

기술 구현



Engine 구조

Render
- Canvas.h
- Renderer.h
- RenderSystem.h
- ScreenBuffer.h

Renderer 비대해짐에 따른 클래스 분할

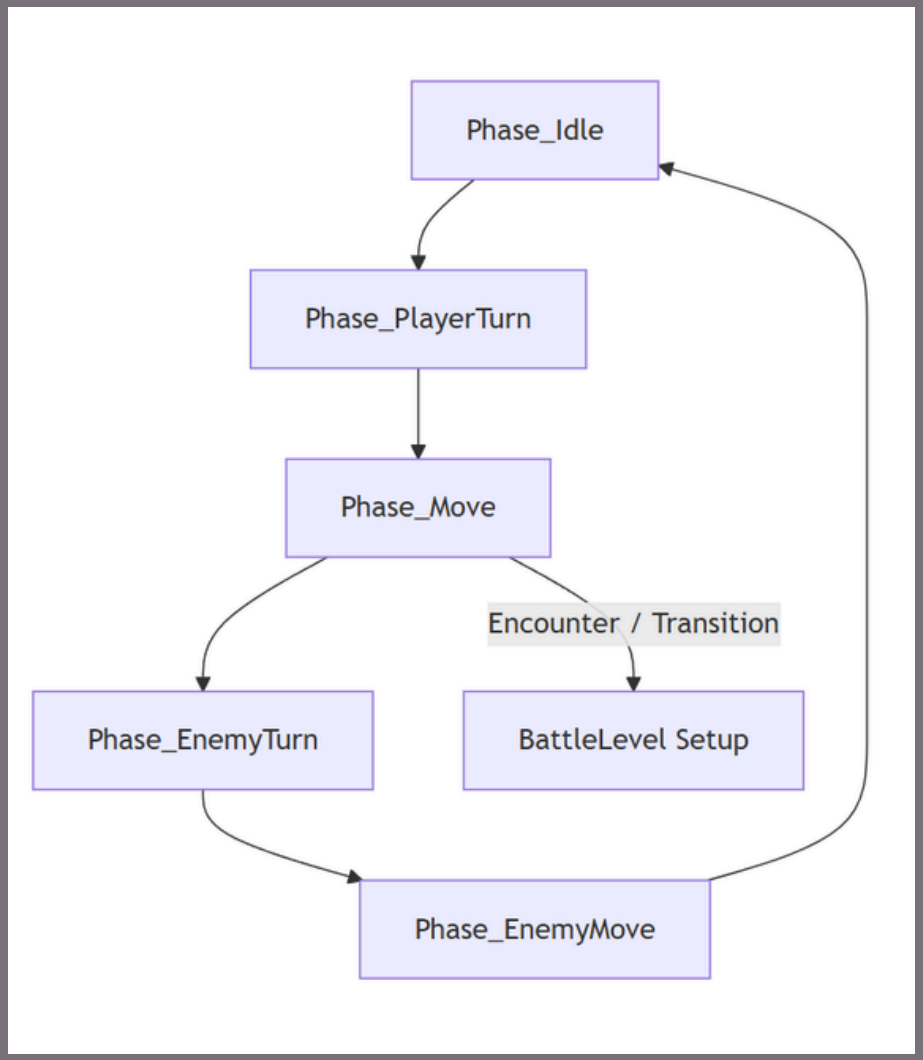
Component
- Component.h
- DisplayComponent.h
- EquipmentComponent.h
- InventoryComponent.h
- SkillComponent.h
- StatComponent.h
- StatusComponent.h

component 증가에 따른 공통 기능, 추상화

Interface
- IBattleCommand.h
- IBattleEventFactory.h
- IBattleLevel.h
- ICutsceneEvent.h
- IEngineCutscenePlayer.h
- IEquipmentData.h
- IRemoveActorCallback.h
- IUIAnimation.h

엔진/ 게임으로 나누면서 필연적으로 참조 역전 발생 => interface

기술 구현



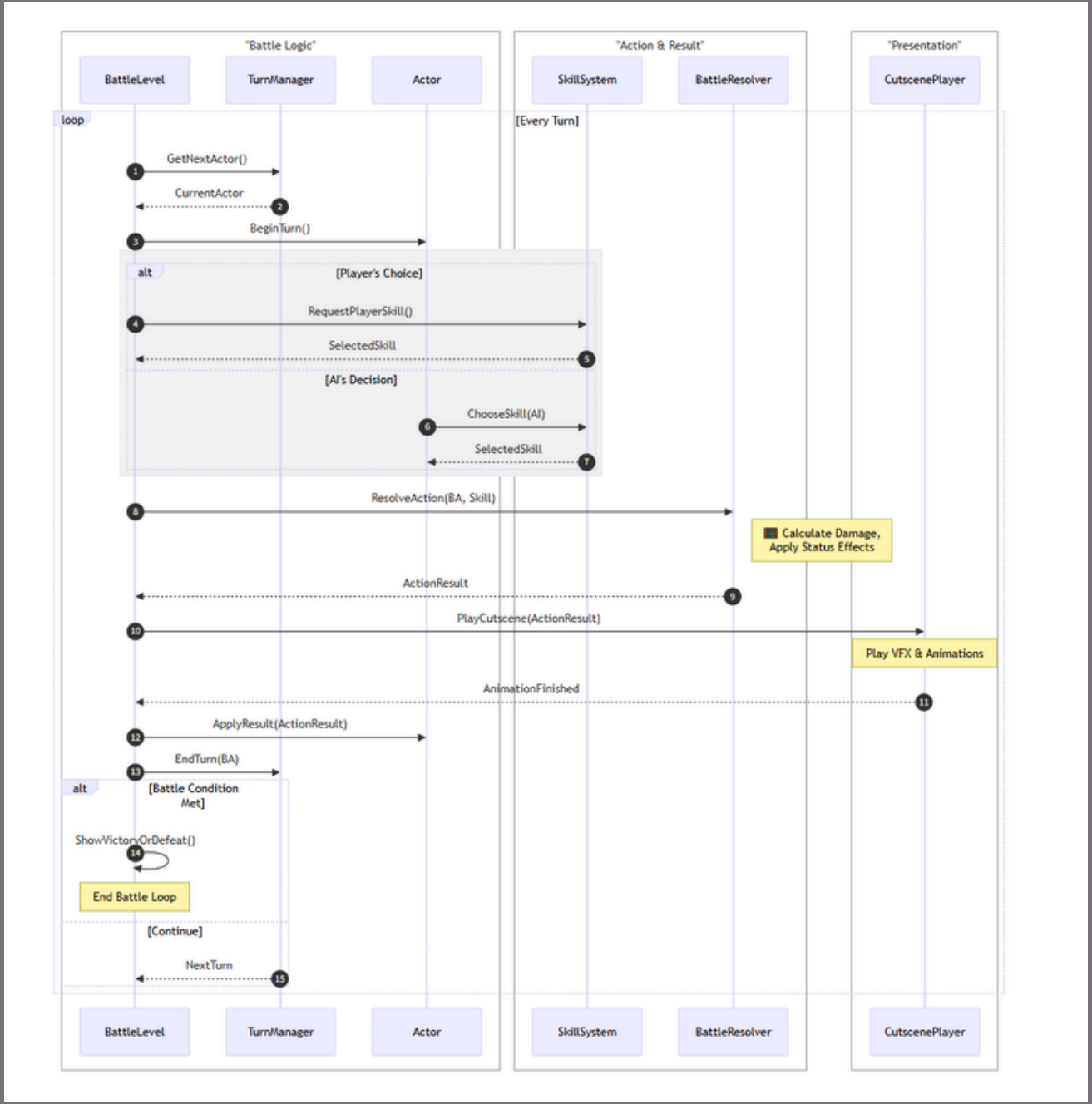
Field 구조

주요 기능
- 이동 범위 계산
- 커서와 충돌시 이동 범위 표기
- monster 충돌시 전투 전환
- portal, level전환

이슈
- level 전환시 메모리 해제
- 길찾기 구현

기술 구현

전투 구조


"Battle Logic"
"Action & Result"
"Presentation"
BattleLevel
TurnManager
Actor
SkillSystem
BattleResolver
CutscenePlayer
loop
[Every Turn]
GetNextActor()
CurrentActor
BeginTurn()
alt
[Player's Choice]
RequestPlayerSkill()
SelectedSkill
[AI's Decision]
ChooseSkill(AI)
SelectedSkill
ResolveAction(BA, Skill)
Calculate Damage, Apply Status Effects
ActionResult
PlayCutscene(ActionResult)
Play VFX & Animations
AnimationFinished
ApplyResult(ActionResult)
EndTurn(BA)
alt
[Battle Condition Met]
ShowVictoryOrDefeat()
End Battle Loop
[Continue]
NextTurn
BattleLevel
TurnManager
Actor
SkillSystem
BattleResolver
CutscenePlayer

전투 구조

입력 → Command→
BattleContext→ Event 생성→CutScenePlayer
BattleLevel tick -> state -> 메뉴 선택 -> command 생성 -> battlecontext 연산 -> event 생성 -> cutsceneplayer 진행-> log, dmg, ascii 등등

Battle
- BattleLevel.h
- BattleContext.h
- Cutsceneplayer.h
- TurnManager.h

BattleLevel, Context 역할/책임 과대
행동 - 계산 - 연출

⇩

TurnManager, atk/skill Calculator, result data 추가

입력→ TurnManager→Command → atk/skill Calculator
→ BattleResultData → CutScenePlayer → EventPlay → World Actor

Battle
- BattleLevel.h
- BattleContext.h
- BattleCalculator.h
- Cutsceneplayer.h
- TurnManager.h

행동 → 계산 → 결과 → 연출 더 나누자.

⇩

BattleResolver, BattleEventProcessor 추가

TurnManager→Command → Context → Resolver
→ Processor → → CutscenePlayer → event...

Battle
- BattleLevel.h
- BattleContext.h
- BattleEventProcessor.h
- BattleLogSystem.h
- BattleResolver.h
- Cutsceneplayer.h
- TurnManager.h

회고

Good	SoSo
프레임 워크 완성	미구현 기능이 눈에 밟힘 ~~인벤, npc, quest, 도감, 영지, 스킬트리, 테이밍, 캐릭터영입, 호감도, 미니맵, 펫, 로그라이크...~~
UI, Effect 구현	할 수 있다 나라면! 마인드의 폐해
전투 로직 구현	
추상화, interface 기능 사용	